



Arabic AI-Text Detection using Apache Spark

By: Abdulaziz Hasan Aljohani

Introduction

- ✓ Developed a **scalable pipeline** to distinguish human-written from AI-generated Arabic abstracts.
- ✓ Utilized **Apache Spark** for distributed processing of massive text datasets.
- ✓ Combined **TF-IDF** semantic vectors with custom **stylometric features**.
- ✓ Implemented **Real-time Stream Processing** simulation

Dataset

14,255
TOTAL SAMPLES

20%
HUMAN WRITTEN

80%
AI GENERATED

4
LLM MODELS

Key Sources

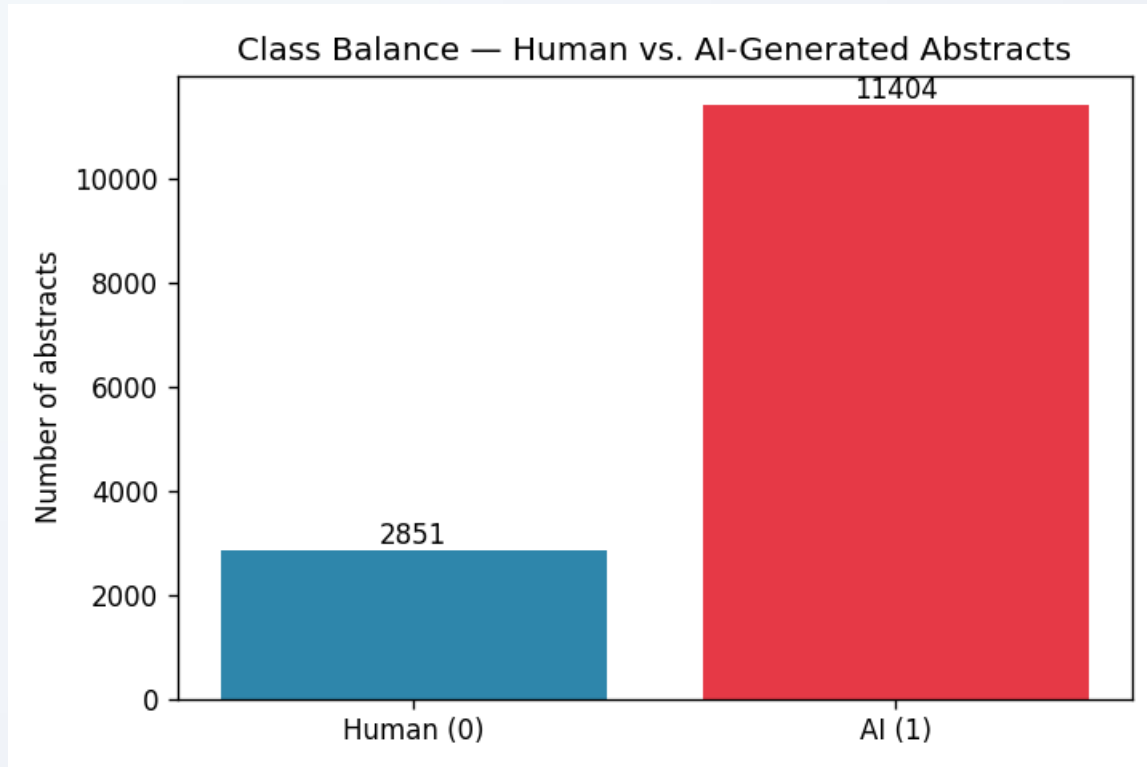
AI samples generated using 4 state-of-the-art models:

Allam & Jais (Arabic Centric)

Llama & OpenAI (General Purpose)

Expletory Data Analysis

Class Balance

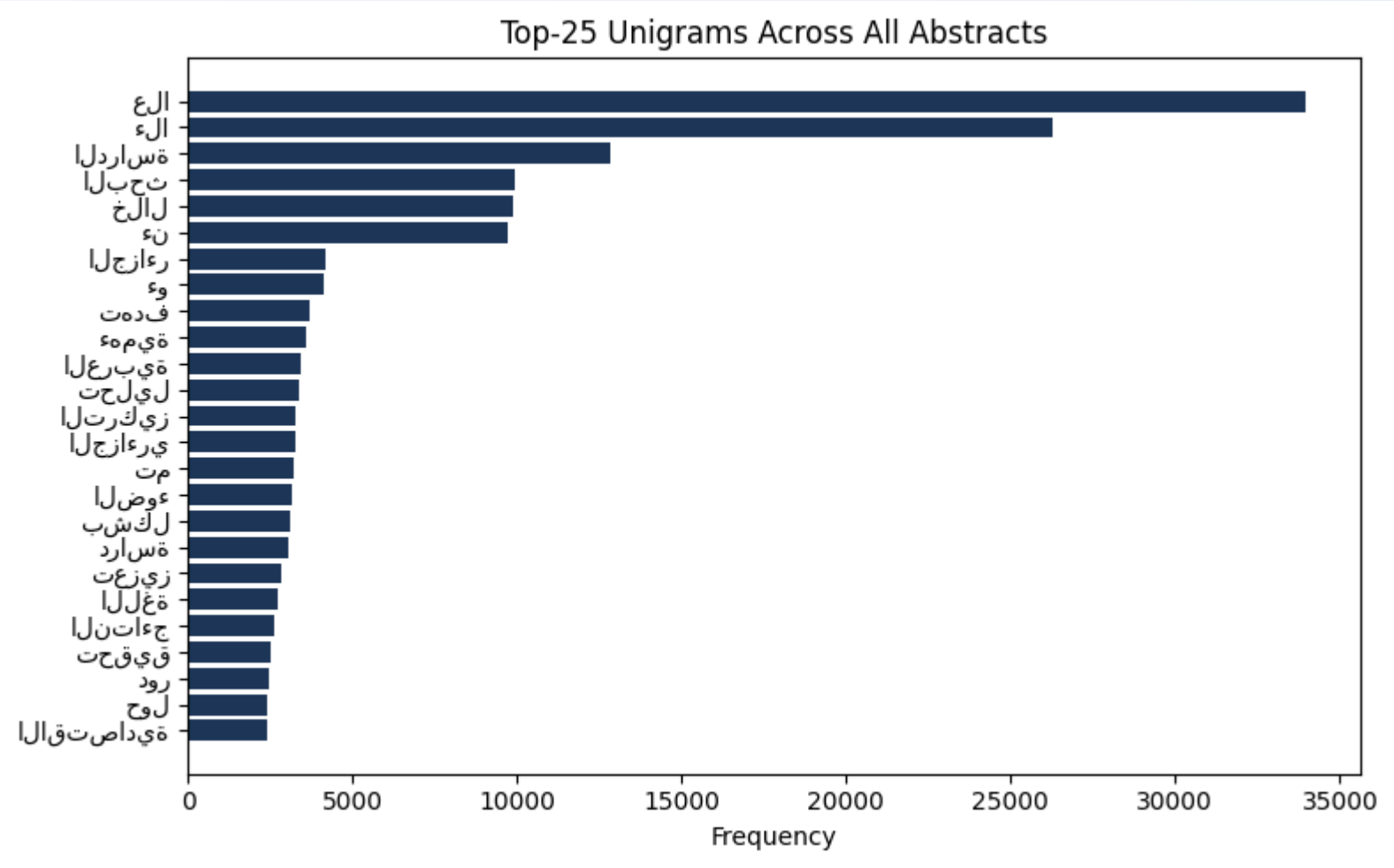


Human 20%

AI 80%

Expletory Data Analysis

Top-25 Unigrams Across All Abstracts



Distributed Preprocessing

Arabic Text Cleaning

- ✓ **Normalization:** Unified Alef/Hamza and removed diacritics using *pyarabic*.
- ✓ **Stopwords:** Broadcasted 701 Arabic stopwords for efficient filtering.
- ✓ **Tokenization:** Segmented text into meaningful units, ignoring short tokens.
- ✓ **Storage limitation:** Optimized via *Parquet* columnar format for Big Data.

Feature Engineering

Stylometric Features

f011: Ratio of long words (>6 chars) to total words.

f034: Physical line count analysis.

Global Features

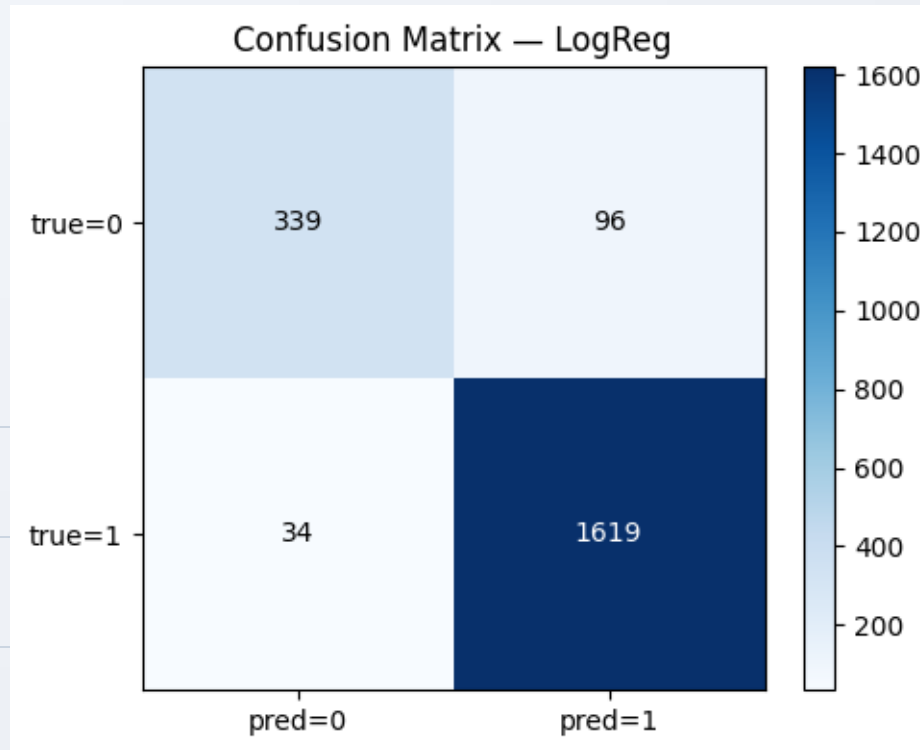
TF-IDF: 4,096-dimensional semantic embedding capturing word importance across the corpus.

Batch Modeling Results

Model	Accuracy	F1-Score	AUC	Train Time
Logistic Regression	0.9435	0.9424	0.9786	8.75s
Naive Bayes	0.8949	0.9009	0.6229	2.15s
Random Forest	0.8130	0.7310	0.9706	13.19s

Insight: Logistic Regression outperformed others, balancing precision and recall effectively.

Confusion Matrix



The model shows high precision for AI detection (1619 correct vs. 34 false positives). It struggles more with false negatives (96 human samples misclassified as AI), indicating a slight bias towards the majority class, which is common in imbalanced datasets

Streaming

- ✓ **Simulation:** JSON file-based stream source to mimic production input.
- ✓ **Spark Structured Streaming:** Unified API for processing live data.
- ✓ **Consistency:** Shared preprocessing pipeline between batch and stream.
- ✓ **Result:** Achieved 100% accuracy on simulated test batches.

Performance & Scalability

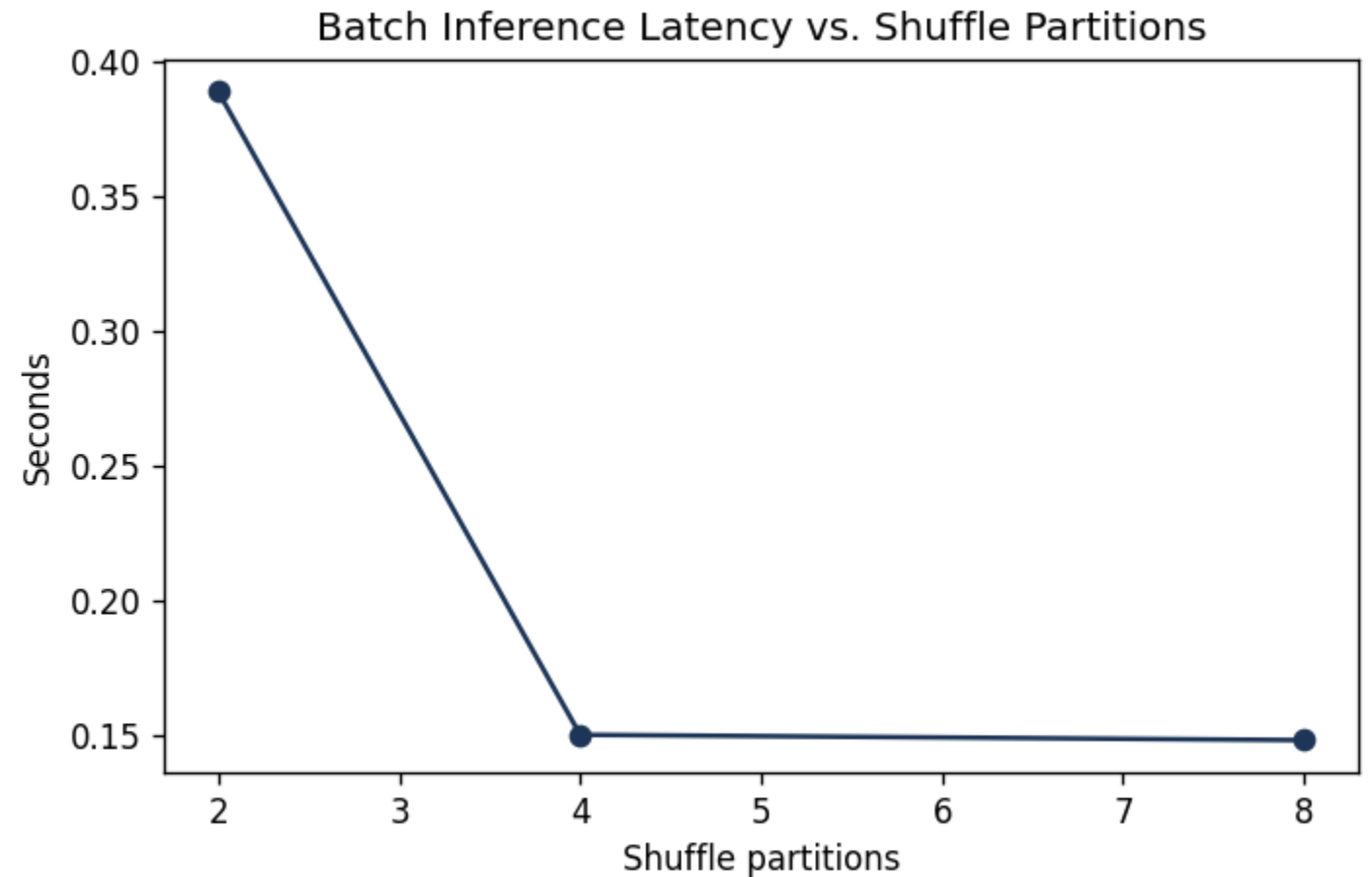
The Power of Parallelism

Inference time on test set (2,088 samples):

2 Partitions: 0.389s

4 Partitions: 0.150s

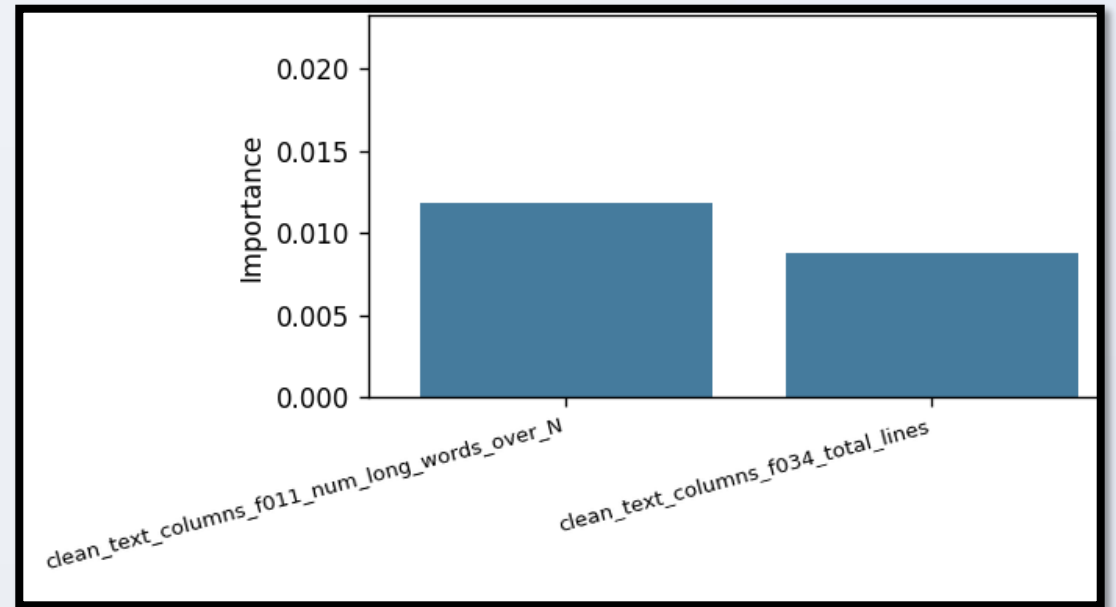
8 Partitions: 0.148s



Feature Importance

The Power of Parallelism

Feature 11 is more important than Feature 34



Conclusion & Future Work

This project apply complete, scalable big data pipeline for Arabic AI-text detection. The Logistic Regression model, trained on a combination of TF-IDF and simple stylometric features, achieved strong performance ($F1 > 0.94$) despite the dataset's imbalance. The implementation of a file-based stream processor validates the architecture's readiness for real-time or near-real-time applications in a production environment

Conclusion & Future Work

Future work

Hybrid Models: Integrate AraBERT for deep semantic understanding.

Attribute Classification: Identify *which* model generated the text.

Multi-domain: Expand to social media and news detection.